Project 3: Implicit Feedback Recommendation Systems

Luke Cabral & Darin Hall
CSDS 435 – Case Western Reserve University

1. Introduction

Recommendation systems are widely used in domains with large item catalogs such as books, beers, and games. In these settings, most interactions take the form of implicit feedback rather than explicit preference labels. The goal of this project is to compare several collaborative filtering approaches using only user–item interaction data. We evaluate three models on three real-world datasets and measure how well each model retrieves a user's held-out relevant item.

We implemented three recommendation models:

1. Alternating Least Squares (ALS)
2. ALS with BM25-based confidence weighting
3. Bayesian Personalized Ranking (BPR)

All three models were trained and evaluated with the implicit library. We compare their performance on BeerAdvocate, Goodreads, and Steam datasets.

2. Problem Definition and Motivation

The input to the problem is a sparse matrix of implicit user interactions with items. Each row corresponds to a user and each column to an item. Non-zero entries indicate the user interacted with that item.

The main goal is to recommend a list of new items for each user. Because explicit ratings are less common in practice, implicit systems are widely used in real applications. The datasets used here also differ in sparsity and domain characteristics, which makes them useful for evaluating how different models behave under different conditions.

3. Methods

3.1 Alternating Least Squares (ALS)

ALS factorizes the user–item interaction matrix into latent user and item factors. The model minimizes a weighted squared error objective with regularization. In our experiments we use:

- 50 latent factors

- 15 iterations
- Regularization parameter 0.1

ALS treats all observed interactions as positive feedback.

3.2 ALS with BM25-Based Confidence

We applied BM25 weighting to adjust the importance of each interaction. The idea is that interactions with extremely popular items are less informative, while interactions with less common items tell us more about a user's taste. We compute BM25 weights on the user–item matrix and convert them to confidence values using:

$c_{ui} = 1 + 10 \times \text{BM25\_weight}_{ui}$

We then train ALS on this reweighted matrix using the same hyperparameters as above.

3.3 Bayesian Personalized Ranking (BPR)

BPR optimizes a pairwise ranking objective. For each user, the model tries to assign a higher score to items the user has interacted with than to items they have not. We use:

- 50 latent factors
- 30 iterations
- Regularization 0.01

BPR is often effective for implicit data, but can struggle when datasets are very sparse or when a strict evaluation protocol is used.

4. Datasets and Cleaning

We used three datasets from the UCSD recommendation dataset collection:

- BeerAdvocate Reviews
- Goodreads Spoilers Ratings
- Steam Reviews

Source: McAuley J. "UCSD Recommender Systems Datasets."
https://cseweb.ucsd.edu/~jmcauley/datasets.html

All datasets were cleaned using the same pipeline:

1. Remove rows missing any required field

2. Remove duplicate user–item interactions
3. Remove users with fewer than 3 interactions
4. Remove items with fewer than 3 interactions
5. Reindex all user and item IDs so they start at 0 and are consecutive

For Steam, the “recommend” flag is converted to a numeric rating:

- True → 5.0
- False → 1.0

4.1 Cleaned Dataset Statistics

Dataset	Interactions	Users	Items	Avg/User	Avg/Item	Sparsity
BeerAdvocate	33,172	3,798	3,963	8.73	8.37	99.78%
Goodreads	27,674	5,933	5,253	4.66	5.27	99.91%
Steam	33,802	7,292	1,637	4.64	20.65	99.72%

Goodreads is the sparsest dataset, followed by BeerAdvocate. Steam has fewer items and many positive-only interactions, which makes it slightly easier to model.

5. Evaluation

We evaluate using a strict per-user leave-one-out protocol:

- For each user with two or more interactions, exactly one item is held out as the test item
- The remaining interactions form the training set
- Models must rank items from the full catalog
- We do not filter training items from the candidate set

Our metrics were Hit Rate at 10 (HR@10), NDCG@10, MAP@10 and Recall@10. Because each user has only one test item and the candidate set includes all items, all four metrics end up having the same numeric values.

6. Results

6.1 BeerAdvocate Results

Model	HR@10	NDCG@10	MAP@10	Recall@10
ALS	0.000268	0.000268	0.000268	0.000268
ALS + BM25	0.000537	0.000537	0.000537	0.000537
BPR	0.000268	0.000268	0.000268	0.000268

ALS with BM25 gave the best performance and improves over plain ALS. BPR ties with ALS on this dataset.

6.2 Goodreads Results

Model	HR@10	NDCG@10	MAP@10	Recall@10
ALS	0.000000	0.000000	0.000000	0.000000
ALS + BM25	0.000203	0.000203	0.000203	0.000203
BPR	0.000000	0.000000	0.000000	0.000000

Goodreads is the most sparse dataset. Only ALS with BM25 recovered any test items in the top 10.

6.3 Steam Results

Model	HR@10	NDCG@10	MAP@10	Recall@10
ALS	0.000614	0.000614	0.000614	0.000614
ALS + BM25	0.001229	0.001229	0.001229	0.001229
BPR	0.000000	0.000000	0.000000	0.000000

Steam is the easiest dataset to model due to its lower item count and binary ratings. ALS with BM25 again gave the best performance.

7. Cross-Dataset Comparison

Across all datasets we observe:

- ALS + BM25 performs best everywhere
- ALS performs second-best
- BPR never outperforms ALS under this evaluation and ties or performs worse

Dataset sparsity strongly affects results. Goodreads has the worst scores for all models, while Steam scores are slightly higher across the board.

8. Parameter Settings

We used the following parameters:

ALS and ALS + BM25:

Factors: 50, Iterations: 15, Regularization: 0.1

BPR:

Factors: 50, Iterations: 30, Regularization: 0.01

BM25 scaling:

alpha = 1, beta = 10

We tested a few nearby parameter values, but none changed the ordering of the models.

9. Conclusion

We implemented and compared three recommendation models using implicit feedback. ALS with BM25-based confidence weighting consistently provided the strongest results across all datasets. BPR did not outperform ALS under our strict evaluation method and often returned no correct items in the top 10.

Even though absolute metric values were small due to the evaluation protocol, the relative differences were consistent. BM25 weighting helped ALS perform better on every dataset, especially in sparse settings.

10. References

[1] Ben Frederickson. *implicit* Python library and documentation. Available at the project repository.

[2] Susan Li. “Building a Recommender System with Implicit Feedback Datasets Using ALS.” Medium, 2020.

[3] Julian McAuley. “Recommender Systems Datasets.” UC San Diego CSE. Public dataset collection page.

[4] J. J. McAuley, J. Leskovec, and D. Jurafsky. “Learning Attitudes and Attributes from Multi Aspect Reviews.” In Proceedings of the IEEE International Conference on Data Mining, 2012. BeerAdvocate multi aspect beer reviews dataset.

[5] M. Wan, J. Misra, R. Nakashima, N. Tajbakhsh, and J. McAuley. “Fine Grained Spoiler Detection from Large Scale Review Corpora.” In Proceedings of the Annual Meeting of the Association for Computational Linguistics, 2019. Goodreads spoilers dataset.

[6] W. Kang and J. McAuley. “Self Attentive Sequential Recommendation.” In Proceedings of the IEEE International Conference on Data Mining, 2018. Steam video game reviews and bundles dataset.